

操作系统

欢迎来到计算机操作系统学习资料分享！此文档是个人在学习操作系统中积累的一些资料和个人总结，希望对看到的人有所帮助，同时也希望有知识、有能力的人一起完善这份文档

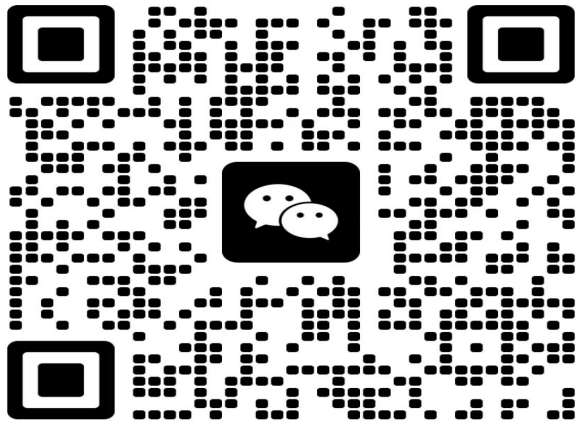
资料分享

相关代码学习

面试八股总结



cpp辅导的阿甘
中国大陆 上海



扫一扫上面的二维码图案，加我为朋友。

本人微信感兴趣的可以加一下，
有什么学习或者求职上的疑惑都可以私信我，
尽量都会给大家解答的，并且周五晚上日常答疑
聊天也会给大家解答



cpp辅导的阿甘 向你推荐

奔跑中的cpp / c++



微信扫码加入星球

这是自己创建的知识星球感兴趣的可以看看，星球介绍可以看下面的链接：
<https://mp.weixin.qq.com/s/INb6nA5ci7GhJMrUTSBSjw>

资料分享

希望看到的人，能够对他的学习、工作有帮助，同时也希望有能力的人可以一起来完善此文档，让它可以发挥出更大的价值。

书籍资料分享

1. UNIX环境高级编程

 [UNIX环境高级编程中文版.pdf](#)

2. 线程池

 [手写线程池与性能分析 .pdf](#)

3. 连接池

 [mysql连接池和redis连接池的实现.pdf](#)

4. Linux命令学习

<https://www.runoob.com/linux/linux-command-manual.html>

https://blog.csdn.net/m0_46422300/article/details/104645072 （记住这上面常用的就可以应付面试了）

（手机下载个Linux命令学习app也可以）

5. 设计模式学习

 [设计模式笔记.docx](#)

6. 常见的系统调用(系统编程)

 [系统编程.zip](#)

7. Linux工具快速教程

 [Linux工具快速教程.pdf](#)

8. Linux命令笔记

 [Linux学习笔记（强悍总结值得一看）.pdf](#)

9. CMake

 [从零开始详细介绍CMake.pdf](#)

10. 鸟哥私房菜

 [鸟哥的linux私房菜.pdf](#)

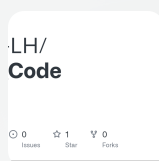
网站分享

1.

相关代码学习

希望看到的人，能够对他的学习、工作有帮助，同时也希望有能力的人可以一起来完善此文档，让它可以发挥出更大的价值。

线程池

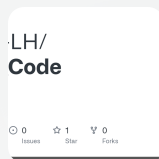


Base_Code/os_code/thread_pool at master · Nikel-LH/Base_Code

Contribute to Nikel-LH/Base_Code development by creating an account on GitHub.

GitHub

内存池

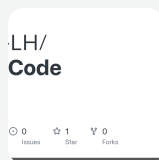


Base_Code/os_code/memory_pool at master · Nikel-LH/Base_Code

Contribute to Nikel-LH/Base_Code development by creating an account on GitHub.

GitHub

内存泄露检测组件



Base_Code/os_code/memoryleak_check at master · Nikel-LH/Base_C...

Contribute to Nikel-LH/Base_Code development by creating an account on GitHub.

GitHub

死锁检测组件

Nikel-LH/
Base_Code

Issues 0 Star 1 Forks 0

Base_Code/os_code/deadlock_check at master · Nikel-LH/Base_Code

Contribute to Nikel-LH/Base_Code development by creating an account on GitHub.

GitHub

Reactor

Nikel-LH/
Base_Code

Issues 0 Star 1 Forks 0

Base_Code/os_code/Reactor at master · Nikel-LH/Base_Code

Contribute to Nikel-LH/Base_Code development by creating an account on GitHub.

GitHub

面试八股总结

希望看到的人，能够对他的学习、工作有帮助，同时也希望有能力的人可以一起来完善此文档，让它可以发挥出更大的价值。

堆空间和栈空间

栈的概念

栈是由操作系统自动分配释放，用于存放函数的参数值、局部变量等。并且函数中定义的局部变量是按照先后定义的顺序依次压入栈的

堆的概念

堆由人为分配和释放，若不进行释放，程序结束时由操作系统进行回收。

两者之间的区别

- **管理方式不同**：栈由操作系统自动分配释放，无需手动释放；堆的申请和释放由人为控制容易产生内存泄露
- **空间大小不同**：每个进程拥有的栈大小要远远小于堆大小。栈的大小一般只有8M
- **生长方向不同**：堆的生长方向向上，内存地址由低到高；栈的生长方向向下，内存地址由高到低（此部分不需要回答）
- **分配方式不同**：堆都是动态分配的，没有静态分配的堆。而栈有**2种分配方式**：静态分配和动态分配。静态分配是由操作系统完成的，比如局部变量的分配。动态分配由`alloca()`函数分配但是无需手动释放，操作系统会进行释放。（重点）
- **分配效率不同**：栈是由操作系统自动分配的，而堆是由c/c++的库函数或运算符来完成申请与管理的，并且频繁的内存申请容易产生内存碎片
- **存放内容不同**：栈存放的内容主要是函数返回的地址、相关参数、局部变量和寄存器等。而堆中具体存放的内容是由程序员填充的。

程序中那些错误发生在堆空间，那些错误发生在栈空间

- 栈：递归函数没有设置；数组越界访问
- 堆：内存泄露；使用已经释放的内存

堆空间和栈空间是用的那个数据结构

- 堆是向高地址扩展的数据结构，是**不连续的内存区域**。是由于系统是用**链表**来存储的空闲内存地址的，自然是不连续的，而链表的遍历方向是由低地址
- 栈是一种**线性表**，并且将线性表的插入和删除操作限制仅为表的一端进行，通常将表中运行进行插入、删除操作的一端称为栈顶

进程

进程与线程的区别

- 进程是资源分配的单位，线程是CPU调度的单位（线程可以单独占用CPU）
- 进程拥有一个完整的资源平台，而线程只独享必不可少的资源，如寄存器和栈
- 线程同样具有就绪、阻塞、执行三种基本状态，同样具有状态之间的转换关系
- 并且进程需要管理一些信息比如文件信息、内存信息，而线程不涉及这些信息的管理，而只是共享它们，因此**线程的创建和销毁是比进程要快的**
- 并且**同一进程之间**的各线程共享内存和文件资源，那么在线程之间数据传递的时候，就不需要进行内核了，这使得线程之间的数据交互效率更高了

父子进程的关系

- fork出来的子进程是父进程的一个复制品，它从父进程处继承了整个进程的地址空间：包括进程上下文（进程执行活动全过程的静态描述）、进程堆栈、打开的文件描述符、信号控

制设定、进程优先级、进程组号等

- 子进程所独有的只有它的进程号，计时器等（只有少量信息）
- Linux的fork（）使用是通过**写时拷贝**（copy-on-write）实现。内核此时并不复制整个进程的地址空间，而是让父子进程共享同一个地址空间。只有在需要写入的时候才会复制地址空间，从而使各个进程拥有各自的地址空间。资源的复制需要写入的时候才会进行，以只读方式共享
- fork之后父子进程**共享文件**，fork产生的子进程与父进程相同的文件描述符指向相同的文件表，引用计数增加，共享文件文件偏移指针
- **重点突出：写时拷贝读时共享**

进程间通信方式

答题要点

同C++的STL中一样答题套路。

建议简历书写**熟悉进程间通信**

面试官看见了肯定会直接问，你都熟悉哪些进程间通信方式啊？

答的话，按照下面给你划分的答就好，稳稳的。答的时间又长，面试管又满意

管道

- 管道分为有名管道。无名管道
- 无名管道其实就是内核里面的一串缓存，它的通信范围是存在父子关系的进程。
- 实际的使用：像linux命令中常用的**管道符（|）**就是无名管道
- 有名管道是一个文件，它的通信可以在不相关的进程间也能相互通信
- 对于管道而言，通信的数据遵循先进先出原则，不支持lseek之类的文件定位操作（改变文件的偏移量）
- 管道这种通信方式效率较低，不适合进程
- -----**说完上面部分，管道介绍就可以结束了**
- 几种场景应对的处理：
 - 如果写端没有关闭，管道中没有数据，这个时候读管道进程去读管道会阻塞；如果写端没有关闭，管道中有数据，这个时候读管道进程会将数据读出，下一次读

没有数据就会阻塞；

- 管道所有写端关闭，读进程去读管道的内容，读取全部内容，最后返回0
- 所有读端没有关闭，如果管道被写满了，写管道进程写管道会被阻塞
- 所有的读端被关闭，写管道进程写管道会收到一个信号，然后退出

消息队列

- 消息队列是保存在内核中的消息链表，在发送数据时，会分成一个一个独立的数据单元，也就是消息（数据块）。固定大小的存储块。
- 消息队列生命周期随内核，如果没有释放消息队列或者没有关闭操作系统，消息队列会一直存在。如果进程从消息队列中读取了消息体，内核就会把这个消息体删除
- -----说完上面两部分就可以了
- 缺点：1.通信不及时；2.附件大小有限制
- 场景：1. 消息队列不适合比较大数据的传输（内核中每个消息体都有一个最大长度的限制）；2. 消息队列通信过程中，存在用户态与内核态之间的数据拷贝开销

共享内存

- 共享内存的机制，就是拿出一块**虚拟地址空间**来，映射到相同的物理内存中。这样这个进程写入的东西，另一个进程可以马上看到，都不需要拷贝来拷贝去，大大提高了通信的速度
- mmap+write () 可以实现零拷贝，减少了一次CPU拷贝。

信号量

- 为了防止多进程共享资源，而造成的数据错乱，所以需要保护机制，使得共享的资源，在任意时刻只能被一个进程访问。正好，信号量就实现了这一保护机制。（这部分不用说）
- 信号量其实就是一个整型的计数器，主要用于实现线程间的互斥与同步，而不是用于缓存进程间通信的数据
- 互斥：为每类共享资源设置一个信号量，初值为1。任何想进入临界区的线程，必须先执行减1操作（sem_wait），在完成对临界资源的访问后再执行加1操作（sem_post）
- 同步：设置两个信号量，一个初值为1，另一个初值为0。先执行的线程，对那个为1的信号量进行减1操作，进行临界资源的访问，然后再对为0的那个信号量进行加1操作。此时另一个线程对这个刚加1的信号量进行减1操作，也进行临界资源的访问，然后对另一个为0的信号量进行加1操作，如此。

信号

- 对于异常情况下工作模式，就需要信号的方式来通知进程
- 信号是进程间通信机制中唯一的异步通信机制
- 比如kill，就是表示给某个进程发送SIGKILL信号
- 用户对信号的几种处理方式：（此部分不说也行）
 - 执行默认操作
 - 捕捉信号
 - 忽略信号
- 使用：
 - 自己对信号的使用的话印象最深的就是对那个僵尸进程的处理吧
 - 僵尸进程
 - 介绍
 - 子进程终止，父进程尚未回收，子进程残留资源存放于内核中，变成僵尸进程
 - 解决方法：
 - 最简单的方法，父进程通过wait和waitpid等函数等待子进程结束，但是，这会导致父进程挂起
 - 如果父进程要处理的事情很多，不能够挂起，通过signal（）函数人为处理信号SIGCHLD，只要有子进程退出自动调用指定好的回调函数，因为子进程结束后，父进程会收到该信号SIGCHLD，可以在其回调函数里调用wait（）或waitpid（）回收
 - 如果父进程不关心子进程什么时候结束，那么可以使用signal通知内核，自己对子进程的结束不感兴趣，父进程忽略此信号，那么子进程结束后，内核会回收，并不再给父进程发送信号

socket

- 本地字节流socket和本地数据报socket在bind的时候，不像TCP和UDP要绑定IP地址和端口，而是绑定一个本地文件

线程

多线程的线程安全问题

线程安全就是多线程访问时，采用了加锁机制，当一个线程访问某个资源时，进行保护，其他线程不能进行访问直到该线程读取完，其他线程才可使用。不会出现数据不一致或者数据污染。线程不安全就是不提供数据访问保护，有可能出现多个线程先后更改数据造成所得到的数据是脏的数据。（记住那个i++操作）

多线程同步的方式

- 锁
 - 互斥锁、自旋锁、读写锁
- 信号量
 - 进程间通信中有描述
- 条件变量
 - 条件变量用于某个线程需要在某种条件成立时才去保护它将要操作的临界区，这种情况从而避免了线程不断轮询检查该条件是否成立而降低效率的情况，这是实现了效率提高。在条件满足时，自动退出阻塞，再加锁进行操作。
 - 条件变量为什么要与互斥锁一起使用呢？（此部分问再说，具体内容可以看下我的博客）
 - 线程4在调用pthread_cond_wait()但线程4还没有进入wait cond的状态的时候，此时线程2调用了 cond_signal 的情况。如果不用mutex锁的话，这个cond_signal就丢失了。加了锁的情况是，线程2必须等到 mutex 被释放（也就是 pthread_cond_wait() 释放锁并进入wait_cond状态，此时线程2上锁）的时候才能调用cond_signal
 - 对其学习，可看如下博客
 - https://blog.csdn.net/weixin_52259848/article/details/127889143?ops_request_misc=%257B%2522request%255Fid%2522%253A%2522e8fc4c50ecace5b0a8d2de5075f5b3c3%2522%252C%2522scm%2522%253A%252220140713.130102334.pc%255Fblog.%2522%257D&request_id=e8fc4c50ecace5b0a8d2de5075f5b3c3&biz_id=0&utm_medium=distribute.pc_search_result.none-task-blog-2~blog~first_rank_ecpm_v1~rank_v31_ecpm-1-127889143-null-null.nonecase&utm_term=%E6%9D%A1%E4%BB%B6%E5%8F%98%E9%87%8F&spm=1018.2226.3001.4450

锁

种类介绍

- 自旋锁pthread_spin_init
 - 当一个线程在获取锁的时候，如果锁已经被其他线程获取，那么该线程将**循环等待（不会睡眠）**，然后不断的判断锁是否能够被成功获取，直到获取到锁才会退出循环
 - 自旋锁是非阻塞锁，**一直占用CPU**，在未获得锁的情况下，一直尝试得到锁，所以占用着CPU，如果不能在很短的时间内获得锁，会使CPU效率降低
 - 场景：锁的内容比较少，阻塞时间较短，可以使用自旋锁
- 互斥锁pthread_mutex_t
 - 互斥锁是**阻塞锁**，当某线程无法获取互斥量时，该线程会被CPU直接挂起，该线程不再消耗CPU，当其他线程释放互斥量后，操作系统会激活那个被挂起的线程，让其投入运行
- 读写锁
 - **读写锁实际是一种特殊的自旋锁**，它把对共享资源的访问者划分成读者和写者，读者只对共享资源进行读访问，写者则需要对共享资源进行写操作。读写互斥，读读共享
 - 多读者可以同时读；写者写时不允许读，不允许其他写者写
 - 读者读写时不允许写者写

死锁介绍（产生条件）

- 死锁现象的介绍（条件）
 - 死锁就是指两个或两个以上的线程（进程）在执行过程中，由于竞争资源或者由于彼此通信而造成的一种阻塞的现象，若在没有外力的作用下，这些线程会一直相互等待，就没办法继续运行，这种情况就是发生了死锁
 - 死锁只有同时满足如下4个条件才会发生
 - 互斥条件：多个线程不能同时使用一个资源
 - 持有并等待条件：线程在占用资源的同时并等待获取其他线程持有的资源
 - 不可剥夺条件：当线程已经持有了资源，在自己使用完之前不能被其他线程获取
 - 环路等待条件：线程 之间获取资源的顺序构成 了环形链

死锁检测组件实现

- 聊到死锁了，建议主动说下，说自己实现了个死锁检测组件，可以放到咱们公司项目代码里，运行看是否有死锁的现象，然后说下面的方法（这也是咱们独有的东西，说好了也绝对是加分项）
- 其实，我们的程序出现死锁，一般就是因为出现了**环路链**。就是A线程在持有资源的同时想获取B线程的资源，B线程在持有资源的时候，想获取C线程的资源，C线程在持有资源的同时又想获取线程A的资源。这样形成了一个环路链，造成了死锁。其实这就是一个有向图环路，那么我们检测程序中是否出现这么一个有向图环路，就可以判断是否出现了死锁
- 首先，我们要构建这个有向图，构建这个有向图的顶点和边。**对于顶点**，要确定锁与线程的对应关系，知道当前的锁被哪个线程占用。在加锁之前，判断这个锁有没有被别的线程使用（**是怎么判断的**），如果没有，在加锁之后我们可以将这个锁与本线程绑定，做一个pair，然后把pair存起来（**怎么保存的**）。比如说，线程 A和mutex1绑定，线程B和mutex2绑定，一个线程和一个锁绑定的组合便是一个顶点。**对于边**，对于线程A再次去尝试对mutex2加锁之前，判断mutex2是否被使用，如果被使用，那么有向图的边就可以确定了，线程A指向mutex2锁被使用的线程B。（怎么实现的）
- 当这个图构建好之后，就可以通过dfs方法来判断是否有环了，进一步检查是否有死锁（**dfs讲讲**）

如何避免死锁

- 避免死锁问题的产生，只要破坏上述四个条件中的一个即可。最常见的并且可行的就是**使用资源有序分配法**，来破坏环路等待条件
- 资源有序分配法：如果有两个线程A和B，这两个线程获取资源的顺序要一样，当线程A是先尝试获取资源A，然后尝试获取资源B的时候，线程B同样也是先尝试获取资源A，然后尝试获取资源B。也就是说，线程A和线程B总是以相同的顺序申请自己想要的资源

零拷贝

○ Linux系统中一切皆文件，仔细想一下Linux系统的很多活动无外乎**读操作和写操作**

○ 传统文件的传输：

- 数据的读取和 写入都是从用户空间到内核空间来回复制。

- 涉及的系统调用：read、write

- 发生了4次用户态与内核态的上下文切换（一个系统调用两次）

- **四次数据拷贝**： 1.DMA拷贝；把磁盘上的数据拷贝到操作系统内核的缓冲区里； 2.CPU拷贝；把内核缓冲区的数据拷贝到用户的缓冲区里； 3.CPU拷贝；把刚才的拷贝到用户的缓冲区里的数据，再拷贝到内核的socket的缓冲区里； 4. DMA拷贝；把内核的socket缓冲区里的数据，拷贝到网卡的缓冲区里 （重点）

○ 传统模式下，涉及多次空间切换和数据冗余拷贝，**效率并不高**。我们可以看到，如果应用程序不对数据做修改，从内核缓冲区到用户缓冲区，再从用户缓冲区到内核缓冲区。两次数据拷贝都需要CPU的参与，并且涉及用户态与内核态的多次切换，加重了CPU负担。我们需要降低冗余数据拷贝、解放CPU，这也就是**零拷贝Zero-Copy技术**。（减少拷贝的次数或减少上下文切换的次数）（这部分总结性简单的说下，不要一个字不拉的说，不然给人一种感觉照着读的感觉）

○ 解决思路：优化文件传输的性能（零拷贝）

- mmap+write

- mmap（）系统调用函数会直接把**内核缓冲区里的数据映射到用户空间（共享缓冲区）**，这样，操作系统内核与用户空间就不需要再进行任何的数据拷贝操作

- 减少了一次CPU拷贝

- sendfile

- **直接把内核缓冲区里的数据拷贝到socket缓冲区里**

- 两次**上下文切换（用户态和内核态的切换）**和三次数据拷贝

- -----（把上面的都答了即可）

- SG-DMA(需要网卡支持)（真正的零拷贝）（此部分面试的时候不说也行）

- 此时sendfile的系统调用会发生变化

- 1.DMA拷贝数据从磁盘到内核缓冲区； 2.缓冲区描述符和数据长度传到socket缓冲区，网卡的SG-DMA控制器**直接将内核缓存中的数据拷贝到网卡的缓冲区里**；此过程不需要将数据从操作系统内核缓冲区拷贝到socket缓冲区中，这样就减少了一次数据拷贝；

- 这就是真的**零拷贝技术**，没有在内存层面去拷贝数据，也就是说**全程没有通过CPU来搬运数据**，所有的数据都是通过DMA来进行传输的

- 零拷贝只需要2次上下文切换和数据拷贝次数，就可以完成文件的传输，而且2次的数据拷贝过程，都不需要通过CPU，2次都是由DMA来搬运

■ 零拷贝的使用

- nginx支持零拷贝技术，一般默认是开启零拷贝技术的（sendfile on）

Reactor

○ 介绍

- reactor其实就是**逆置了事件处理流程**，应用程序需要提供相应的**接口并注册到reactor上**，如果相应的事件发生，reactor将主动调用应用程序注册的接口（**回调函数**）。其实感觉reactor就是对**epoll（管理的）**进行封装，进行网络IO与业务的解耦，将epoll管理IO变成管理事件，整个程序由事件进行驱动执行

○ 构成：reactor主要就是三部分组成

- 多路复用器：select、poll、epoll（管理器）
- 事件分发器：将多路复用器中返回的**就绪事件分到对应的处理函数中**（对应事件fd与对应回调函数绑定）
- 事件处理器：负责处理特定事件的处理函数（具体**回调函数**的处理逻辑读写提取）

○ 处理流程：

- （1）注册相应的事件处理器（将开始listen监听fd注册到就绪事件）；（2）多路复用器等事件；（3）事件到来，激活事件分发器，**分发器调用事件到对应的处理器**；（4）事件处理器处理事件，然后注册新的事件（比如读事件，完成读操作后，根据业务处理数据，注册写事件，写事件根据业务响应请求；比如listen读事件，肯定要给新的连接注册读事件）

○ reactor搭配非阻塞io的原因：（此部分问再说，单纯介绍reactor不需要说）

- 多线程环境（每个线程中都有个reactor对象）：有连接listenfd到来了，多个epoll进行检测，如果listenfd为阻塞的，**其中一个线程的accept提取了，则其他线程的accept就会一直阻塞（惊群现象）**
- 边缘触发：读事件触发时，read在一次事件循环中把read buffer读空
- select bug（reactor的io多路检测模型不一定用的是epoll，还可能是select）：当某个socket接收缓冲区有新数据字节到达，然后select报告这个socket描述符可读，但随后，协议栈检查到这个新字节检验和错误，然后丢弃这个字节，这时候调用read则无数据可读，如果socket没有被设置nonblocking，此read将阻塞当前线程。

○ 单reactor多线程：其实就是对事件处理的时候，引入了个线程池呗，别的没啥不一样的

多路复用器（select/epoll/poll）

- select是将文件描述符用fd_set的变量传递给内核(数组传递)，由内核进行检查将就绪的文件描述符置1返回，再由用户态遍历进行判断处理；其主要特点是内核拷贝和两次遍历导致性能开销大，同时监听的文件描述符受限于fd_set的大小，一般为1024
- poll和select类似，不同之处在于poll在用户态通过数组传递文件描述符，在内核态转为链表存储
- epoll对select和poll进行了改进，避免了性能开销大和文件描述符数量少的特点，其用红黑树来存储文件描述符，将就绪的文件描述符放到一个队列中，每个文件描述符只需在添加时传入一次，通过事件来更改文件描述符状态；epoll中为每个文件描述符指定回调函数，在就绪时将其加入到就绪队列，调用epoll时候只需要判断队列是否为空即可
- 使用场景:select适用于文件描述符少且多数活跃的场景，如果文件描述符少也用epoll反而开销过大；而epoll适用于文件描述符很多的场景
- -----此部分把这三个复用器存储fd的存储结构说下就可以了

池式组件

线程池

- 首先，是有一个线程组,即有多个工作线程。即在有一个队列，来存储任务（此线程池实现的比较简单，你也可以定义两个队列，一个存储任务的队列，一个存储执行任务的队列）。当有任务到来，即调用获取任务的函数，此时把任务存储到一个队列中，然后唤醒一个线程（pthread_cond_signal（）），处理队列中的任务（执行对应的回调函数）。当全部程序运行结束，退出线程池时，释放分配的空间，销毁线程即可
- 虚假唤醒问题（此部分问再说也行，也可以提前说看自己掌握）
 - 任务队列为空且线程池未退出时，进行睡眠判定用的是while循环而不用if
 - 原因：
 - 避免虚假唤醒：linux pthread_cond_signal（以前版本可能会唤醒两个线程），被唤醒后不一定有任务，虚假唤醒
 - linux 可能被信号唤醒
 - 业务逻辑不严谨，被其他线程抢了该任务（假设有两个线程，其中一个是活跃的，另一个是休眠的。假设有任务到来，活跃的线程取了任务，但是也唤醒了那个休眠的线程，如果此处不是while，那它就会向下执行报错）

连接池

- 池化技术能够减少资源对象的创建次数；数据库连接池（Connection pooling）是**程序启动时建立足够的数据库连接**，并将这些连接组成一个连接池，由程序动态地对池中的连接进行申请，使用，释放。
- 逻辑：创建多个连接对象，用一个**队列存起来（空闲队列）**，当要用连接对象操作数据库的时候，就把存起来的对象拿出来用（获取连接），并把它移除，放到存储使用对象的容器中（**使用队列**）。当用完它之后，就把它从使用队列中移除，放到空闲队列中，等着让其他想要操数据库的线程接着使用，避免了频繁的创建和销毁。当系统运行完毕，把所有创建的对象统一销毁即可
- 获取连接的思路：（1）空闲连接大于0，获取连接返回；（2）空闲连接为0，当前连接数小于最大连接数时，新建连接并插入空闲队列，获取连接返回；当前连接数为最大连接数，根据传的 **计量时间（timeout_ms）** 来判断，如果 $timeout_ms \leq 0$ ，则表明我选择**死等的方式，即一直阻塞等待，直到有空闲连接为止。否则， $timeout_ms > 0$ ，则表示等待阻塞这么长时间，如果超过了这么长时间，还么空闲连接，直接返回NULL即可。

内存池

实现：我这个是内存池是按照大小块进行存储的（设置了个阈值判定，比如4k）。内存池会**预申请**一块4k的内存块，当**用户向内存池申请小于4k时**，内存池会从这内存块的空间中划出size空间，并且进行引用计数加1。再有申请，依旧划分并且计数加1。如果这个内存块剩余空间**不足以分配size大小时**，此时内存池会**再次申请一块内存块**，再从新的内存块划分size空间给用户。每次释放小内存时，都会在内存块中引用计数减1，**只有当引用计数为0的时候，才会回收内存块使他重新成为空闲的空间，以便重复利用空间**。这样，内存池避免频繁向内核申请/释放内存，从而提高了系统性能。对于**大块内存的话**，内存池**不会预先**申请内存，用户申请的时候，内存池再申请内存，然后返回给用户。并且对于大块，回收就直接free。

简历书写

针对总结的这部分资料来说，简历书写建议如下内容：

- 熟悉进程通信、零拷贝、了解死锁以及简单死锁检测实现和单线程 reactor 简单实现、对于池式组件线程池、连接池、内存池以及内存泄露检测简单实现有所了解

阿甘(42379064)

阿甘(42379064)